

Methodenüberschreibung (Overriding)

Kriterium	Beschreibung	Wann zu verwenden?
Definition von Methodenüberschreibung	Methodenüberschreibung tritt auf, wenn eine Methode in einer Unterklasse dieselbe Signatur (Name, Parameter) wie eine Methode in der Oberklasse hat, aber mit einer anderen Implementierung.	Wenn das Verhalten einer Methode in einer Unterklasse geändert oder spezifisch angepasst werden soll, um das Verhalten der Oberklasse zu modifizieren.
Vererbung	Methodenüberschreibung ist nur möglich, wenn die Methode in der Oberklasse vorhanden ist und von der Unterklasse geerbt wird.	Wenn eine Methode in einer Oberklasse definiert ist, die von einer Unterklasse geerbt wird und das Verhalten der Methode geändert werden muss.
Polymorphismus	Methodenüberschreibung ermöglicht Polymorphismus. Wenn eine Methode in einer Oberklasse überschrieben wird, kann die zur Laufzeit aufgerufene Methode je nach tatsächlichem Objekttyp (nicht des Variablentyps) variieren.	Wenn dasselbe Verhalten in einer Hierarchie mit unterschiedlichen Implementierungen für verschiedene Klassen ausgeführt werden soll.
Verwendung von <code>@Override</code> Annotation	In Java kann die <code>@Override</code> -Annotation verwendet werden, um sicherzustellen, dass eine Methode korrekt überschrieben wird und keine Tippfehler oder falsche Signaturen vorliegen.	Um die Fehlerwahrscheinlichkeit zu verringern und sicherzustellen, dass die Methode korrekt überschrieben wird.
Zugriffsmodifikatoren	In der überschriebenen Methode kann der Zugriffsmodifikator in der Unterklasse nicht restriktiver sein als in der Oberklasse. Wenn eine Methode in der Oberklasse <code>public</code> ist, muss sie auch in der Unterklasse <code>public</code> sein.	Wenn eine Methode überschrieben werden muss und der Zugriffsmodifikator beachtet werden muss.
Rückgabewert	Der Rückgabewert der überschriebenen Methode muss kompatibel mit dem Rückgabewert der Methode in der Oberklasse sein. In Java kann der Rückgabewert jedoch auch ein Untertyp des ursprünglichen Rückgabetyps sein (Covariant Return Type).	Wenn die Rückgabetypen auf der Unterklasse basieren sollen, um eine spezifischere Rückgabe zu erhalten.

Kriterium	Beschreibung	Wann zu verwenden?
Verwendung in Vererbungshierarchien	Methodenüberschreibung wird oft in Vererbungshierarchien verwendet, um das Verhalten der Basisklasse in den abgeleiteten Klassen zu spezifizieren.	Wenn eine Klasse das Verhalten einer Basisklasse spezifizieren und anpassen muss, um funktionale Anforderungen zu erfüllen.
Dynamisches Binden (Late Binding)	Die Entscheidung, welche Methode ausgeführt wird, erfolgt zur Laufzeit, basierend auf dem tatsächlichen Objekttyp, und nicht zur Kompilierzeit.	Wenn unterschiedliche Implementierungen zur Laufzeit basierend auf dem konkreten Objekttyp aufgerufen werden sollen.
Vererbung vs. Überschreibung	Während Vererbung es einer Unterklasse ermöglicht, von einer Oberklasse zu erben, bietet die Überschreibung die Möglichkeit, das Verhalten einer geerbten Methode zu ändern.	Wenn das Verhalten einer Methode der Oberklasse angepasst oder spezifiziert werden muss, um die Anforderungen der Unterklasse zu erfüllen.
Verwendung von <code>super</code>	Die Methode der Oberklasse kann durch <code>super</code> innerhalb der überschriebenen Methode aufgerufen werden, um auf die Implementierung der Oberklasse zuzugreifen.	Wenn das Verhalten der Methode der Oberklasse beibehalten, aber erweitert oder verändert werden soll.
Verwendung von abstrakten Methoden	Wenn eine Methode in der Oberklasse als <code>abstract</code> deklariert ist, muss sie in der Unterklasse überschrieben werden.	Wenn eine Klasse eine abstrakte Methode definiert und die Unterklasse das Verhalten dieser Methode spezifizieren muss.
Verwendung in Entwurfsmustern	Methodenüberschreibung wird häufig in Entwurfsmustern wie Strategy, Template Method und Command verwendet, um das Verhalten von Klassen zur Laufzeit zu ändern.	Wenn das Verhalten einer Klasse flexibel und erweiterbar bleiben muss, ohne dass bestehender Code geändert wird.
Überladen vs. Überschreiben	Methodenüberladung kann innerhalb einer Klasse mit verschiedenen Parametern verwendet werden, aber Methodenüberschreibung betrifft die gleiche Signatur der Methode in der Unterklasse.	Wenn die Methodensignatur unverändert bleibt, aber das Verhalten in der Unterklasse geändert werden soll.
Verwendung bei abstrakten Klassen	Eine abstrakte Klasse kann abstrakte Methoden enthalten, die von ihren Unterklassen überschrieben werden müssen.	Wenn eine abstrakte Klasse mit abstrakten Methoden als Basisklasse verwendet wird, die von ihren konkreten Unterklassen überschrieben werden müssen.

Kriterium	Beschreibung	Wann zu verwenden?
Verwendung bei finalen Methoden	Eine Methode, die in der Oberklasse als <code>final</code> deklariert ist, kann nicht überschrieben werden.	Wenn eine Methode in der Oberklasse nicht überschrieben werden soll und ihre Implementierung als endgültig festgelegt wird.

Zusammenfassung

Methodenüberschreibung ist besonders geeignet, wenn:

- Eine Methode aus einer Oberklasse angepasst oder spezifiziert werden muss, um das Verhalten der Unterklasse zu erweitern oder zu ändern.
- Polymorphismus benötigt wird, um dieselbe Methode für verschiedene Objekttypen mit unterschiedlichen Implementierungen zu verwenden.
- Eine abstrakte Methode definiert wurde, die in der Unterklasse überschrieben werden muss, um funktionale Anforderungen zu erfüllen.
- Es notwendig ist, das Verhalten einer geerbten Methode zu verändern oder zu erweitern, ohne den gesamten Code in der Oberklasse zu ändern.

Vorteile

- **Flexibilität**
Methoden in der Unterklasse können die geerbte Logik der Oberklasse überschreiben, um das Verhalten der Instanz zu spezifizieren.
- **Polymorphismus**
Durch die Verwendung von überschriebenen Methoden können unterschiedliche Implementierungen zur Laufzeit dynamisch gewählt werden.
- **Erweiterbarkeit**
Die Funktionalität der Oberklasse kann in der Unterklasse spezifiziert oder angepasst werden, ohne dass der bestehende Code in der Oberklasse geändert werden muss.

Nachteile

- **Verwirrung**
Bei zu vielen Überschreibungen in tief verschachtelten Hierarchien kann es schwierig sein, das Verhalten einer Methode nachzuvollziehen, besonders bei komplexen Vererbungshierarchien.
- **Fehleranfälligkeit**
Wenn Methoden falsch überschrieben oder mit falschen Parametern aufgerufen werden, kann dies zu Laufzeitfehlern führen, besonders bei falschem Polymorphismus.